Database Management Systems Laboratory

Mini-Project Report

VibeCheck

Audio Recognition and Discovery System

Submitted by

MIS No.	Name
612415061	Bhavesh Gadling
612415070	Jibran Jamdar
612415095	Sangram Lohokare

October 2026

Contents

1 Problem Statement **2**

2 Objectives **2**

3 Functional Requirements **2**

3.1 Functional requirements 2

3.2 Non-functional requirements 3

3.3 Technology used 3

4 ER Diagram **3**

4.1 Entities and attributes 3

4.2 Relationships 4

5 Relational Schemas Obtained from ER **6**

5.1 Mapping rules applied 6

5.2 Relations 6

5.3 Foreign keys 6

5.4 Data types and constraints 7

6 Functional Dependencies on Each Table **8**

7 Normalization up to 3NF **9**

7.1 Definitions used 9

7.2 How the design was reached 9

7.3 Normal form of each relation 11

7.4 Controlled redundancy: `tot_songs` 11

7.5 Conclusion 11

A Database Objects Used by the Application **11**

A.1 Indexes 11

A.2 Trigger to keep `tot_songs` correct 12

A.3 Stored function to log a recognition 12

A.4 Views 12

A.5 Sample queries for demonstration 13

1 Problem Statement

People often hear a song in a café, in a film or on another device and want to know its title and artist immediately. Searching by half-remembered lyrics is slow and often fails. This project builds a small database application that listens to a few seconds of audio and tells the user which song is playing, along with its artist, genre and release year.

Audio cannot be compared sample by sample, because a phone recording differs from the original in loudness, background noise and starting point. The system therefore converts audio into an *audio fingerprint*: a set of integer hashes, each derived from a pair of spectral peaks, together with the time at which the pair occurs in the song. The fingerprints of every song known to the system are stored in a relational database. A recording is identified by looking up its hashes in the database and checking which song has many matching hashes that line up consistently in time. The identified song is saved in the user's recognition history.

The database problem is therefore to store a catalogue of songs, artists, genres, users and recognition history, together with a large table of fingerprints, so that a short noisy clip can be matched to the right song quickly while all data stays consistent.

2 Objectives

1. Design a relational database for users, songs, artists, genres, audio fingerprints and recognition history, normalized up to Third Normal Form.
2. Store fingerprints in a way that makes matching fast, using a B-Tree index on the hash column.
3. Maintain data integrity with primary keys, foreign keys, unique and check constraints, and cascading deletes.
4. Keep derived data consistent automatically using a trigger, and make every recognition an atomic transaction.
5. Provide database views that hide joins and aggregation, so the application reads song details and user history with a simple query.
6. Build a working front end (mobile app) and back end (REST API) on top of the database, so the design is tested by a real application.

3 Functional Requirements

3.1 Functional requirements

ID	Requirement
FR1	A new user can register with first name, optional middle and last name, e-mail address and password.
FR2	A registered user can log in with e-mail and password and receives an authentication token.
FR3	A logged-in user can record about six seconds of audio on the phone and send it to the server.
FR4	The server converts the audio to a fingerprint, matches it against the stored fingerprints, and returns the result.
FR5	For a match, the system shows the title, artist(s), genre(s) and release year of the song.

ID	Requirement
FR6	Every successful recognition is saved in the user's history as part of the same database transaction.
FR7	A user can view the history of recognized songs, newest first.
FR8	Songs can be added to the catalogue from audio files. The system stores the song, its artist and genre.
FR9	The number of songs of each artist is kept up to date automatically when songs are linked to or unlinked from an artist.

3.2 Non-functional requirements

- **Speed:** hash lookups use an index, so matching does not scan the whole fingerprint table.
- **Integrity:** invalid data (negative durations, malformed e-mails, unknown roles) is rejected by the database itself.
- **Security:** passwords are salted and hashed with PBKDF2, and the API is protected by tokens.
- **Consistency:** each recognition request runs in one transaction that is committed on success and rolled back on error.

3.3 Technology used

Layer	Technology
Database	PostgreSQL
Back end	Python, FastAPI, psycopg2, NumPy, SciPy, pydub (ffmpeg)
Front end	React Native with Expo (Android phone)

4 ER Diagram

4.1 Entities and attributes

Entity	Attributes	Key attribute
User	user_id, name (composite: first_name, middle_name, last_name), email, password_hash, created_at	`user_id`
Song	song_id, title, duration, release_year, audio_file_url, created_at	`song_id`
Artist	artist_id, name (composite: first_name, middle_name, last_name), stage_name, dob, nationality, tot_songs (derived), awards, bio	`artist_id`
Genre	genre_id, name, description	`genre_id`
Fingerprint	fp_id, hash, time_offset	`fp_id`

4.2 Relationships

Relationship	Entities	Cardinality	Participation	Meaning
Recognizes	User – Song	M : N	partial on both sides	A user recognizes songs; the same user may recognize the same song many times. Attribute: `recognized_at`.
Performs	Artist – Song	M : N	partial on both sides	An artist performs many songs and a song can have several artists. Attribute: `role` (Primary, Featured, Producer).
Belongs to	Song – Genre	M : N	partial on both sides	A song can belong to several genres and a genre contains many songs.
Has	Song – Fingerprint	1 : N	total on Fingerprint, partial on Song	A song has many fingerprint hashes; every fingerprint belongs to exactly one song.

Notes. (i) `tot_songs` is a derived attribute: it equals the number of songs of the artist, and is shown with a dashed oval. (ii) Because the same user can recognize the same song many times, each instance of *Recognizes* needs its own identity, so a surrogate key `history_id` is used when the relationship is converted to a table.

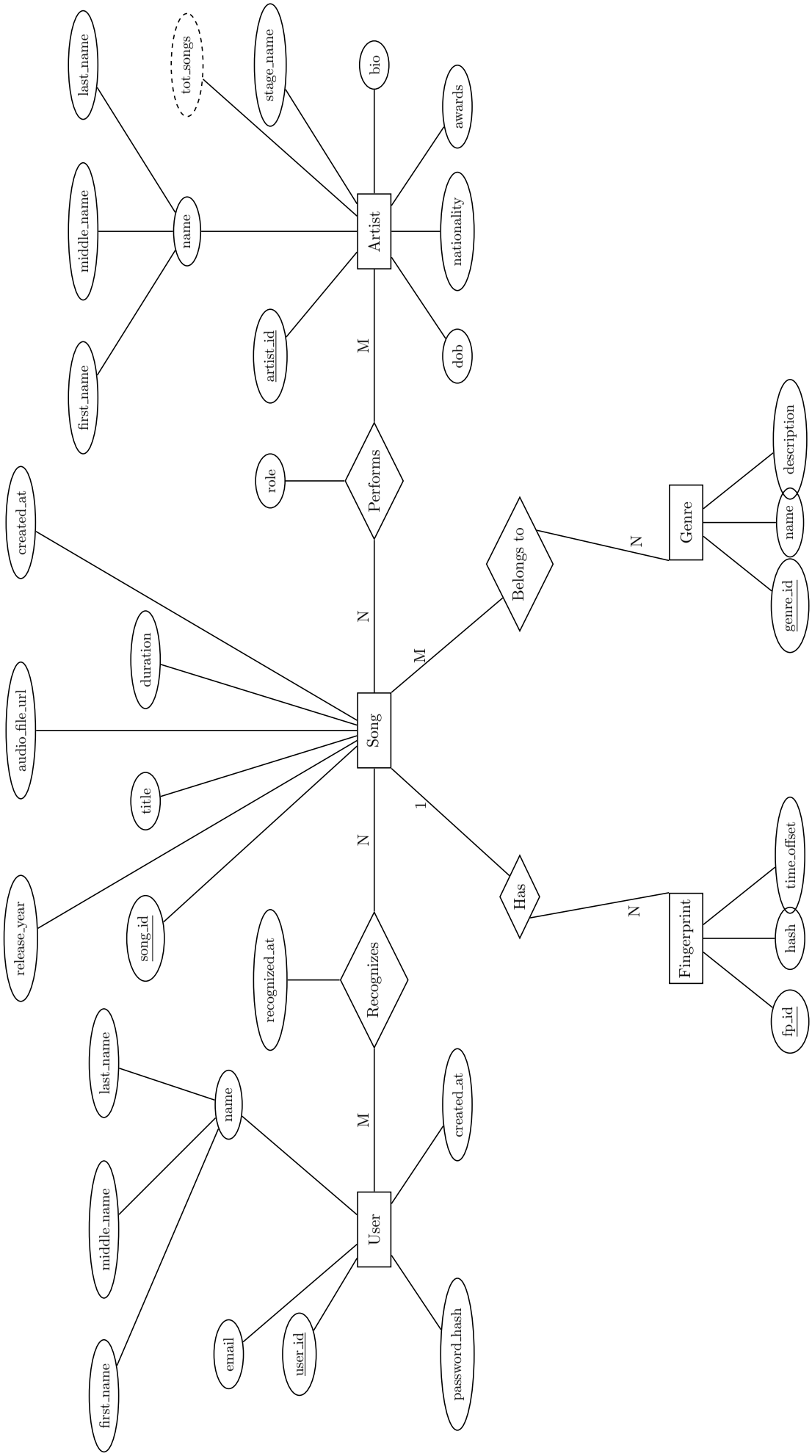


Figure 1: Conceptual Entity-Relationship Diagram of VibeCheck

5 Relational Schemas Obtained from ER

5.1 Mapping rules applied

1. Each strong entity becomes a relation whose attributes are the simple attributes of the entity. A composite attribute is replaced by its components (`name` becomes `first_name`, `middle_name`, `last_name`). The key attribute becomes the primary key.
2. Each M:N relationship becomes a separate relation containing the primary keys of both participating entities (as foreign keys, together forming the primary key) plus the attributes of the relationship.
3. Each 1:N relationship is represented by adding the primary key of the "one" side to the "many" side as a foreign key.
4. The derived attribute `tot_songs` is stored as an ordinary column and kept correct by a trigger (see Section 7.4).

5.2 Relations

Primary keys are underlined; foreign keys are listed below the relations.

- **Users**(user_id, first_name, middle_name, last_name, email, password_hash, created_at)
- **Artists**(artist_id, first_name, middle_name, last_name, stage_name, dob, nationality, tot_songs, awards, bio)
- **Genres**(genre_id, name, description)
- **Songs**(song_id, title, duration, release_year, created_at)
- **Song_Artists**(song_id, artist_id, role)
- **Song_Genres**(song_id, genre_id)
- **Fingerprints**(fp_id, song_id, hash, time_offset)
- **History**(history_id, user_id, song_id, recognized_at)

5.3 Foreign keys

All foreign keys are declared with `ON DELETE CASCADE`, so deleting a parent row removes its dependent rows.

Child relation	Foreign key	References
Song_Artists	song_id	Songs(song_id)
Song_Artists	artist_id	Artists(artist_id)
Song_Genres	song_id	Songs(song_id)
Song_Genres	genre_id	Genres(genre_id)
Fingerprints	song_id	Songs(song_id)
History	user_id	Users(user_id)
History	song_id	Songs(song_id)

5.4 Data types and constraints

Attribute	Data type	Constraints
Users		
user_id	SERIAL	Primary key, auto-increment
first_name	VARCHAR(50)	NOT NULL
middle_name	VARCHAR(50)	optional
last_name	VARCHAR(50)	optional
email	VARCHAR(255)	UNIQUE, NOT NULL, CHECK for a valid e-mail pattern
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT current time
Artists		
artist_id	SERIAL	Primary key, auto-increment
first_name, middle_name, last_name	VARCHAR(50)	optional
stage_name	VARCHAR(100)	UNIQUE, NOT NULL
dob	DATE	CHECK date of birth is in the past
nationality	VARCHAR(50)	optional
tot_songs	INT	DEFAULT 0, CHECK ≥ 0
awards, bio	TEXT	optional
Genres		
genre_id	SERIAL	Primary key, auto-increment
name	VARCHAR(50)	UNIQUE, NOT NULL
description	TEXT	optional
Songs		
song_id	SERIAL	Primary key, auto-increment
title	VARCHAR(150)	NOT NULL
duration	INT	NOT NULL, CHECK > 0 (seconds)
release_year	INT	CHECK ≥ 1900
created_at	TIMESTAMP	DEFAULT current time
Song_Artists		
song_id	INT	Part of primary key, FK to Songs, ON DELETE CASCADE
artist_id	INT	Part of primary key, FK to Artists, ON DELETE CASCADE
role	VARCHAR(50)	DEFAULT 'Primary', CHECK in (Primary, Featured, Producer)
Song_Genres		
song_id	INT	Part of primary key, FK to Songs, ON DELETE CASCADE
genre_id	INT	Part of primary key, FK to Genres, ON DELETE CASCADE

Attribute	Data type	Constraints
Fingerprints		
fp_id	BIGSERIAL	Primary key, auto-increment
song_id	INT	NOT NULL, FK to Songs, ON DELETE CASCADE, indexed
hash	BIGINT	NOT NULL, B-Tree index
time_offset	INT	NOT NULL, CHECK ≥ 0 (frame number in the song)
History		
history_id	BIGSERIAL	Primary key, auto-increment
user_id	INT	NOT NULL, FK to Users, ON DELETE CASCADE
song_id	INT	NOT NULL, FK to Songs, ON DELETE CASCADE
recognized_at	TIMESTAMP	DEFAULT current time; index on (user_id, recognized_at descending)

6 Functional Dependencies on Each Table

The following functional dependencies must hold in every valid state of the database. They follow from the meaning of the attributes and from the declared keys. Right-hand sides written as "all other attributes" list every non-key attribute of the table.

Relation	Candidate key(s)	Functional dependencies
Users	user_id; email	user_id → first_name, middle_name, last_name, email, password_hash, created_at email → user_id
Artists	artist_id; stage_name	artist_id → first_name, middle_name, last_name, stage_name, dob, nationality, tot_songs, awards, bio stage_name → artist_id
Genres	genre_id; name	genre_id → name, description name → genre_id
Songs	song_id	song_id → title, duration, release_year, audio_file_url, created_at
Song_Artists	(song_id, artist_id)	(song_id, artist_id) → role
Song_Genres	(song_id, genre_id)	Only the trivial dependencies; the whole row is the key
Fingerprints	fp_id	fp_id → song_id, hash, time_offset
History	history_id	history_id → user_id, song_id, recognized_at

Assumptions. Two different songs may have the same title, so title does not determine song_id. Likewise a hash value can occur in many songs and at many times, so hash determines nothing else, and role depends on the combination of a song and an artist, not on either alone. Different users can recognize the same song, and one user can recognize it repeatedly, so user_id and song_id together do not determine history_id.

7 Normalization up to 3NF

7.1 Definitions used

- **1NF:** all attribute values are atomic; there are no repeating groups or multi-valued columns.
- **2NF:** the relation is in 1NF and no non-prime attribute depends on only part of a composite candidate key.
- **3NF:** the relation is in 2NF and, for every non-trivial dependency $X \to A$, either X is a superkey or A is a prime attribute (so no non-prime attribute depends transitively on a key).

7.2 How the design was reached

The design was reached in stages, starting from flat tables that hold everything. For every stage the relations, their keys and the dependencies used are listed; primary keys are underlined. Only the attributes needed to show each step are included, and the remaining columns of Users, Artists, Songs and so on are added in the final schema.

Stage 0: unnormalized

All information about a song is kept in one table, with repeating groups (written in braces):

> **SongInfo0**(song_id, title, duration, release_year, {artist_id, stage_name, nationality, role}, {genre_id, genre_name})

A song can have several artists and several genres, so one row would have to hold a list of artists and a list of genres. This is not even in 1NF.

Stage 1: first normal form (1NF)

Repeating groups are removed by giving every combination of song, artist and genre its own row, so all values become atomic. The recognition side of the system is also kept as flat tables at this stage:

- **SongInfo**(song_id, artist_id, genre_id, title, duration, release_year, stage_name, nationality, role, genre_name)
- **RecognitionLog**(history_id, user_id, first_name, email, song_id, title, recognized_at)
- **FingerprintLog**(fp_id, song_id, title, hash, time_offset)

The dependencies that hold are:

- song_id → title, duration, release_year
- artist_id → stage_name, nationality
- (song_id, artist_id) → role
- genre_id → genre_name
- history_id → user_id, song_id, recognized_at; user_id → first_name, email; song_id → title
- fp_id → song_id, hash, time_offset; song_id → title

Problems: the title and duration of a song are repeated for every artist and genre combination, an artist cannot be stored before having a song, and deleting the last song of an artist would erase the artist's data.

Stage 2: second normal form (2NF)

In SongInfo the key has three attributes, but `title`, `duration`, `release_year`, `stage_name`, `nationality` and `genre_name` each depend on only a part of the key, and `role` depends on two of the three. These partial dependencies are removed by splitting SongInfo into one relation per dependency:

- **Song**(`song_id`, `title`, `duration`, `release_year`)
- **Artist**(`artist_id`, `stage_name`, `nationality`)
- **Genre**(`genre_id`, `genre_name`)
- **SongArtist**(`song_id`, `artist_id`, `role`)
- **SongGenre**(`song_id`, `genre_id`)
- **RecognitionLog**(`history_id`, `user_id`, `first_name`, `email`, `song_id`, `title`, `recognized_at`) (unchanged)
- **FingerprintLog**(`fp_id`, `song_id`, `title`, `hash`, `time_offset`) (unchanged)

Now every non-key attribute depends on the whole key of its relation. RecognitionLog and FingerprintLog have single-attribute keys, so they cannot have partial dependencies and are already in 2NF.

Stage 3: third normal form (3NF)

RecognitionLog and FingerprintLog are in 2NF but not in 3NF, because they contain transitive dependencies:

- `history_id` → `user_id` → `first_name`, `email`
- `history_id` → `song_id` → `title`
- `fp_id` → `song_id` → `title`

If a user changed an e-mail address, every one of their history rows would have to change, and the title of a song would be repeated in every fingerprint row of that song (tens of thousands of times). The transitive dependencies are removed by keeping only the foreign key and moving the other attributes to the relation that owns them:

- **History**(`history_id`, `user_id`, `song_id`, `recognized_at`)
- **Users**(`user_id`, `first_name`, `email`, ...)
- **Fingerprints**(`fp_id`, `song_id`, `hash`, `time_offset`)
- **Songs**(`song_id`, `title`, `duration`, `release_year`, ...) (the Song relation of Stage 2)

Together with Artists, Genres, Song_Artists and Song_Genres from Stage 2 (with their remaining columns added), these are the eight relations of Section 5.

Stage	Relations after the stage
0 (unnormalized)	SongInfo0
1 (1NF)	SongInfo, RecognitionLog, FingerprintLog
2 (2NF)	Song, Artist, Genre, SongArtist, SongGenre, RecognitionLog, FingerprintLog
3 (3NF)	Users, Artists, Genres, Songs, Song_Artists, Song_Genres, Fingerprints, History

7.3 Normal form of each relation

Relation	1NF	2NF	3NF	Justification
Users	Yes	Yes	Yes	All values are atomic (the name is split into three columns). The key `user_id` is a single attribute, so there are no partial dependencies. Every determinant (`user_id`, `email`) is a candidate key.
Artists	Yes	Yes	Yes	Single-attribute key; the only determinants are `artist_id` and `stage_name`, both candidate keys. No non-prime attribute determines another.
Genres	Yes	Yes	Yes	The determinants `genre_id` and `name` are both candidate keys.
Songs	Yes	Yes	Yes	The only determinant is the key `song_id`; artist and genre data are not stored here.
Song_Artists	Yes	Yes	Yes	The only non-key attribute, `role`, depends on the whole key (`song_id`, `artist_id`), not on a part of it.
Song_Genres	Yes	Yes	Yes	All attributes belong to the key, so there are no non-prime attributes.
Fingerprints	Yes	Yes	Yes	Single-attribute key `fp_id`; `song_id`, `hash`, and `time_offset` describe the fingerprint row.
History	Yes	Yes	Yes	Single-attribute key `history_id`; `user_id`, `song_id`, and `recognized_at` describe one recognition event and do not determine each other.

In each relation every determinant is a candidate key, so all eight relations are also in Boyce–Codd Normal Form.

7.4 Controlled redundancy: `tot_songs`

The column `Artists.tot_songs` is derived, because it equals the number of rows of Song_Artists for that artist. Storing it is a deliberate choice that makes it cheap to display an artist's song count without a join and aggregation. To prevent the stored value from becoming inconsistent, two triggers on Song_Artists increase the count after an insert and decrease it after a delete (never below zero). The value is therefore never edited by hand, and the design stays consistent while avoiding repeated counting.

7.5 Conclusion

All eight tables satisfy 1NF, 2NF and 3NF (and BCNF). Each fact is stored once, many-to-many relationships are resolved through the linking tables Song_Artists and Song_Genres, and consistency is enforced by keys, check constraints, cascading foreign keys and a trigger.

A Database Objects Used by the Application

A.1 Indexes

```
CREATE INDEX idx_fingerprints_hash ON Fingerprints(hash);
CREATE INDEX idx_fingerprints_song ON Fingerprints(song_id);
CREATE INDEX idx_history_user ON History(user_id, recognized_at DESC);
```

The index on `hash` makes the matching query `WHERE hash = ANY(...)` an index scan instead of a full table scan. The history index serves the query that lists a user's latest recognitions.

A.2 Trigger to keep `tot_songs` correct

```
CREATE OR REPLACE FUNCTION update_artist_song_count() RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    UPDATE Artists SET tot_songs = tot_songs + 1 WHERE artist_id = NEW.artist_id;
    RETURN NEW;
  ELSIF TG_OP = 'DELETE' THEN
    UPDATE Artists SET tot_songs = GREATEST(tot_songs - 1, 0) WHERE artist_id = OLD.
        artist_id;
    RETURN OLD;
  END IF;
  RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_increment_tot_songs AFTER INSERT ON Song_Artists
  FOR EACH ROW EXECUTE FUNCTION update_artist_song_count();
CREATE TRIGGER trg_decrement_tot_songs AFTER DELETE ON Song_Artists
  FOR EACH ROW EXECUTE FUNCTION update_artist_song_count();
```

A.3 Stored function to log a recognition

```
CREATE OR REPLACE FUNCTION log_recognition(p_user INT, p_song INT) RETURNS BIGINT AS
    $$
DECLARE new_id BIGINT;
BEGIN
  INSERT INTO History(user_id, song_id) VALUES (p_user, p_song) RETURNING history_id
      INTO new_id;
  RETURN new_id;
END; $$ LANGUAGE plpgsql;
```

The API calls this function in the same transaction in which it reads the matched song, so the history entry and the response are consistent.

A.4 Views

```
CREATE OR REPLACE VIEW v_song_details AS
SELECT s.song_id, s.title, s.duration, s.release_year,
       COALESCE(string_agg(DISTINCT a.stage_name, ', '), 'Unknown') AS artists,
       COALESCE(string_agg(DISTINCT g.name, ' / '), 'Unclassified') AS genres
FROM Songs s
LEFT JOIN Song_Artists sa ON sa.song_id = s.song_id
LEFT JOIN Artists a ON a.artist_id = sa.artist_id
LEFT JOIN Song_Genres sg ON sg.song_id = s.song_id
LEFT JOIN Genres g ON g.genre_id = sg.genre_id
GROUP BY s.song_id;

CREATE OR REPLACE VIEW v_user_history AS
SELECT h.history_id, h.user_id, h.recognized_at, d.*
FROM History h JOIN v_song_details d ON d.song_id = h.song_id;
```

A.5 Sample queries for demonstration

```
EXPLAIN ANALYZE SELECT * FROM Fingerprints WHERE hash = 123;
SELECT * FROM v_user_history;
SELECT stage_name, tot_songs FROM Artists;
```